Stoquify verification lanes

Reviewed 7 October 2026. Owner: engineering/QA. This is a repository command contract, not a plugin configuration format. Read WORKFLOW.md before selecting evidence for a change.

Command selection

Run `node scripts/verify-agent-workflow-contract.js` to check reviewed script text, absent/present npm lifecycle hooks, normalized SHA-256 fingerprints of execution helpers, and required project references. Run `node scripts/verify-agent-workflow-contract.js --command agent:workflow:test` to classify an exact package script. The classifier never executes that script. Unknown names, changed commands, changed helpers, added hooks, and prohibited audit selections fail closed.

The default gate checks a small reviewed manifest, not every package script. It reports how many scripts are outside it; adding an unrelated script does not freeze the entire repository. Selecting that new script still fails until reviewed. `auditAllowed` means eligible for the documented local verification lane, not no filesystem writes: Jest can write temporary cache and fixtures. Additional arguments, environment overrides, direct shell commands, installation lifecycle scripts, npm configuration, dependency changes and arbitrary shell delegation are outside this classification. Review them separately. The helper performs no npm install, network access, application setup, database query or migration.

The manifest is an integrity tripwire, not semantic analysis or a sandbox. It recognizes simple `npm run` / `npm test` delegates; a maintainer must inspect the complete execution chain before admitting a command. Audit-allowed entries currently prohibit all npm pre/post hooks. Do not approve a changed command by blindly regenerating hashes. Inspect its diff, local helper imports, config/setup, lifecycle hooks and installed tool behavior, then update only the reviewed entries in command-registry.json. Hashes normalize CRLF to LF for cross-platform checkouts. Application modules under test remain editable without maintaining fingerprints for the whole application; their import and side effects still require review when changed.

Local lanes

Lane	Invocation	Evidence and limits
Workflow contract	`npm run agent:workflow:check`	Read-only comparison of repository-owned workflow references and reviewed command inputs. Does not certify security or hook execution.
Workflow contract fixtures	`npm run agent:workflow:test`	Isolated Node/Jest tests via `scripts/jest.agent-workflow.config.cjs`; creates/removes its own OS temp fixtures. No Next config, dotenv loading, app setup or DB.
Bounded ECC pilots	`npm run test:ecc:pilots`	`jest.ecc-pilots.config.cjs` selects ten inspected auth, inventory, purchasing, accounting, payroll, POS, public-color and component suites; persistence/actions are mocked or source files are inspected. `nextJest()` without `dir` avoids application Next config, dotenv and tsconfig loading. Config, setup and the ten test files are fingerprinted. No browser or PostgreSQL certification.

Lane	Invocation	Evidence and limits
Existing application unit/component tests	Explicit `--runTestsByPath` after inspecting the named test, setup and imports	Default `jest.config.ts` loads `next/jest` with `dir: './'`, hence Next config and environment. `jest.setup.ts` mocks Prisma, router and inventory utilities. Passing results are mocked/unit evidence; `.integration.test.ts` names alone do not establish database integration. Use the bounded pilot configuration where its approved exact suites cover the change.
Database concurrency/recovery	Only the separately reviewed disposable-target commands below	Real writes and fixture cleanup. Not allowed in audit mode and not run by the workflow contract.
Browser journeys	Existing approved browser configuration and explicit fixture account/target	Separate from jsdom component results. Record actual URL, viewport, role, fixture, screenshot/trace and failing step. Do not infer browser verification from passing source tests.

`build`, `verify:ci`, migration commands, seeds and their aliases are excluded from audit work. In particular `build` delegates to `prisma:migrate:deploy:safe`; `verify:ci` invokes `prisma:migrate:deploy`. `policy:gates` also contains DB preparation and runtime certification. A word such as `safe`, `gate`, `test` or `preflight` does not establish read-only behavior. Unlisted aliases are refused by classification.

Inspected domain evidence routes

These are a test inventory, not an assertion that all imports or database prerequisites are approved for execution. The audit-allowed pilot manifest defines the small automatically eligible subset. Commands below are exact runner forms for a future scoped check, after reviewing its target and current source. No credentials belong in this document or committed evidence.

Domain	Mocked/isolated example inspected	Actual PostgreSQL route and prerequisites
Auth/tenant authority	`lib/security/__tests__/server-authz.test.ts` mocks RBAC and module entitlement; `lib/security/__tests__/rbac.test.ts` is in the bounded pilot lane.	No dedicated auth-session revocation PostgreSQL runner established by this slice. Unit permission checks do not prove persisted session invalidation; schedule explicit fixture/integration and browser evidence for that claim.
AP/payment recovery	`services/purchasing/__tests__/payment-posting-recovery.service.test.ts` mocks `$transaction`, authority, release evidence and accounting posting. `payment-release-evidence.test.ts` is selected by the pilot config.	`npm run test:purchase-receiving:postgres` executes `scripts/run-purchase-receiving-postgres-tests.js` for `services/purchase-order/__tests__/purchase-receiving.postgres.test.ts`. It reads `DATABASE_URL` or `.env`, requires a local host, drops/recreates `codex_purchase_receiving_cert_20260818`, executes `prisma db push --skip-generate --accept-data-loss`, applies receiving migration SQL and drops the schema in cleanup. This is receiving evidence, not AP payment-release/recovery certification. A separately scoped AP recovery race/rollback database suite remains a prerequisite for that claim.

Domain	Mocked/isolated example inspected	Actual PostgreSQL route and prerequisites
POS checkout	`services/pos/__tests__/checkout-lifecycle.service.test.ts` mocks `$transaction`, cart authority and commit lookup.	`node node_modules/jest/bin/jest.js --config jest.payroll-trust-spine.config.cjs --runInBand --runTestsByPath services/pos/__tests__/checkout-lifecycle.postgres.test.ts`, with explicitly reviewed `DATABASE_URL` on `localhost`, schema `codex_pos_lifecycle_20260920`, and `RUN_POS_LIFECYCLE_POSTGRES=1`. Test creates its schema, copies existing public table/enum definitions, creates fixtures, then drops the created schema. Requires current public tables. Without the flag the suite skips; a skip is not evidence.
POS full financial commit	`services/pos/__tests__/commit-recovery.service.test.ts` is a separate unit suite to inspect for a recovery change.	`node node_modules/jest/bin/jest.js --config jest.payroll-trust-spine.config.cjs --runInBand --runTestsByPath services/pos/__tests__/pos-full-commit.postgres.test.ts`, with `RUN_POS_FULL_COMMIT_POSTGRES=1`, `POS_DISPOSABLE_DATABASE_NAME` matching `stoquify_pos_audit_YYYYMMDD_<hex>`, and explicit `DATABASE_URL` on `localhost` pointing to that DB with `schema=public`. Requires prepared schema/columns; creates synthetic organizations, records and failure-injection triggers/sequences. Its zero-tax fixture is not statutory certification. Do not combine with an inferred existing application target.
Payroll settlement/immutability	`services/payroll/__tests__/payroll-payment-reconciliation.service.test.ts` mocks Prisma transactions, sensitive-action decisions, business events and close invalidation.	`npm run payroll:trust-spine:postgres` first invokes `payroll:immutability:prepare` and the reset helper, then `scripts/payroll-trust-spine-postgres-certification.js` through `scripts/with-payroll-immutability-test-db.js`. Require explicit dedicated `PAYROLL_IMMUTABILITY_DATABASE_URL` or `TEST_DATABASE_URL`, inspect reset constraints, migrate and generate prerequisites before use. The wrapper can derive from `.env`; this workflow must not rely on that fallback. Preparation can reset a test DB; certification writes race/rollback fixtures and Markdown/JSON evidence.

The alternate existing `jest.ui-workflow-postgres.config.cjs` has no Prisma mock setup and targets `tests/postgres/ui-workflow/`, including `payroll-settlement.test.ts` and `purchase-authoring.test.ts`. Its bootstrap `node scripts/ui-workflow-postgres-bootstrap.cjs --database stoquify_local_ui_workflow_<unique-name>` creates a new local database, runs migrations and writes rehearsal evidence. It is a mutation lane, not an audit prerequisite. Review its target helper and `scripts/prisma-local-fresh-bootstrap.js` before execution; do not invoke it just to run the new workflow validator.

Choosing the right proof

For auth changes, report permission/revocation cases and which persistence boundaries were mocked. For inventory, payroll, payments and POS, distinguish unit invariants from actual transaction rollback, uniqueness and concurrent contenders on PostgreSQL. For UI work, report keyboard, loading/error/empty/recovery and localization evidence separately from screenshots. For every run retain command, suite names, counts, target identity without secrets, date and source baseline; list not-run checks and skipped suites explicitly.

The default coverage collection in `jest.config.ts` historically omits `services/`. A dedicated services coverage lane should report the selected service domains without implying whole-platform coverage. Adding coverage collection does not add missing database tests or turn mocked tests into database evidence. The implementation report records the separately integrated coverage scope and actual run status.

Maintenance and rollback

On changes to package commands, their pre/post hooks or reviewed execution helpers, review the relevant manifest entry in the same PR. On skill updates, verify that suggested commands still resolve through this contract. Monthly, review stale references and whether the pilot suite still covers the intended risks; no recurring automation is activated by this document. The workflow owner maintains the manifest; domain owners maintain their invariants and runtime evidence. Revert a broken new gate as a scoped reviewed change; preserve the existing financial, migration and release gates.